

Manual: Java y MongoDB

Mario Sánchez Barroso

2º CFGS DAM

IES Profesor Juan Antonio Carrillo Salcedo

Índice

1. Preparación del Entorno.....	3
1.1 Despliegue de MongoDB con Docker	3
1.2 Configuración del Proyecto Maven	4
2. Componente de la Conexión	4
3. Pruebas de Funcionamiento (CRUD)	6
Resultados de Ejecución	7
4. Conclusiones	7
5. Bibliografía	7

Tecnologías Utilizadas:

Lenguaje de Programación: Java

Gestor de Dependencias: Maven

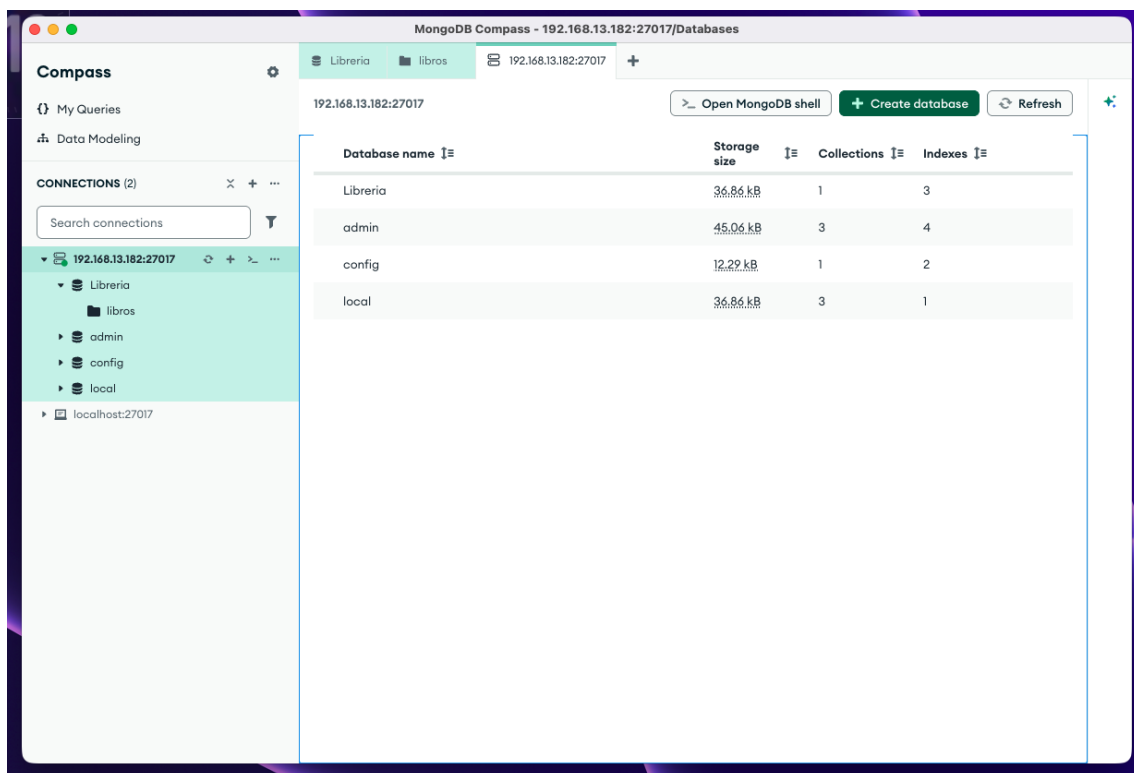
IDE: Eclipse

Driver: MongoDB Java Driver (Sync)

1. Preparación del Entorno

1.1 Despliegue de MongoDB con Docker

En este caso tengo instalado el contenedor Docker en el servidor Ubuntu de promox, podemos comprobarlo ya que en el Mongo Compass tenemos la conexión establecida



Puerto: 27017 (Puerto por defecto).

Usuario: root

Contraseña: ad1234

1.2 Configuración del Proyecto Maven

En Eclipse, crea un nuevo proyecto Maven. Abre el archivo `pom.xml` y añade la siguiente dependencia dentro de la etiqueta `<dependencies>`:

```
<dependency>

    <groupId>org.mongodb</groupId>

    <artifactId>mongodb-driver-sync</artifactId>

    <version>5.0.0</version>
</dependency>

<dependency>

    <groupId>ch.qos.logback</groupId>

    <artifactId>logback-classic</artifactId>

    <version>1.4.14</version>
</dependency>
```

2. Componente de la Conexión

Crearemos una clase llamada `MongoConnection` que encapsulará la lógica para conectarse a la base de datos.

Archivo: MongoConnection.java

```
1 package pack;
2
3 import com.mongodb.MongoException;
4 import com.mongodb.client.MongoClient;
5 import com.mongodb.client.MongoClients;
6 import com.mongodb.client.MongoDatabase;
7
8 public class MongoConnection {
9
10     private static final String CONNECTION_STRING = "mongodb://root:ad1234@192.168.13.182:27017";
11     private static final String DATABASE_NAME = "mi_empresa";
12
13     private MongoClient mongoClient;
14     private MongoDatabase database;
15
16     public MongoConnection() {
17         try {
18             this.mongoClient = MongoClients.create(CONNECTION_STRING);
19             this.database = mongoClient.getDatabase(DATABASE_NAME);
20
21             this.database.runCommand(new org.bson.Document("ping", 1));
22
23             System.out.println(">> Conexión a MongoDB exitosa: " + DATABASE_NAME);
24         } catch (MongoException e) {
25             System.err.println(">> Error conectando a MongoDB: " + e.getMessage());
26             throw e;
27         }
28     }
29
30     public MongoDatabase getDatabase() {
31         return database;
32     }
33
34     public void close() {
35         if (this.mongoClient != null) {
36             this.mongoClient.close();
37             System.out.println(">> Conexión cerrada.");
38         }
39     }
40 }
```

3. Pruebas de Funcionamiento (CRUD)

Realizaremos las operaciones básicas sobre una colección llamada empleados.

Archivo: MainTests.java

```
2
3 import com.mongodb.client.MongoCollection;
9
10 public class MainTests {
11     public static void main(String[] args) {
12         // 1. Iniciar conexión
13         MongoConnection connection = new MongoConnection();
14         MongoDBDatabase db = connection.getDatabase();
15
16         // Obtener la colección (tabla)
17         MongoCollection<Document> collection = db.getCollection("empleados");
18
19         System.out.println("--- INICIO DE PRUEBAS CRUD ---");
20
21         // CREATE (Insertar)
22         Document nuevoEmpleado = new Document()
23             .append("nombre", "Carlos Perez")
24             .append("puesto", "Desarrollador Junior")
25             .append("salario", 25000)
26             .append("activo", true);
27
28         collection.insertOne(nuevoEmpleado);
29         System.out.println("Documento insertado. ID: " + nuevoEmpleado.get("_id"));
30
31         // READ (Leer)
32         System.out.println("Buscando empleado 'Carlos Perez:");
33         Document encontrado = collection.find(Filters.eq("nombre", "Carlos Perez")).first();
34
35         if (encontrado != null) {
36             System.out.println("    Datos: " + encontrado.toJson());
37         }
38
39         // UPDATE (Actualizar)
40         System.out.println("Actualizando salario y puesto...");
41         collection.updateOne(
42             Filters.eq("nombre", "Carlos Perez"),
43             Updates.combine(
44                 Updates.set("salario", 30000),
45                 Updates.set("puesto", "Desarrollador Mid")
46             )
47         );
48
49         // Verificación de actualización
50         Document actualizado = collection.find(Filters.eq("nombre", "Carlos Perez")).first();
51         System.out.println("    Dato actualizado: " + actualizado.toJson());
52
53         // DELETE (Borrar)
54         System.out.println("Eliminando empleado...");
55         collection.deleteOne(Filters.eq("nombre", "Carlos Perez"));
56
57         // Verificación de borrado
58         long count = collection.countDocuments(Filters.eq("nombre", "Carlos Perez"));
59         System.out.println("    Empleados restantes con ese nombre: " + count);
60
61         // Cerrar conexión
62         connection.close();
63     }
64 }
```

Resultados de Ejecución

Al ejecutar la clase `MainTests`, la consola mostrará la siguiente salida, evidenciando el éxito de las operaciones:

Consola de Salida:

```
Console X
<terminated> MainTests (1) [Java Application] /Users/alumnado/p2/pool/plugins/org.eclipse.justi.openjdk.hotspot.jre.full.macosx.x86_64_21.0.8.v20250724-1412/jre/bin/java (9 feb 2026, 13:23:45 - 13:23:48 elapsed 0:00:02) [pid: 58130]
>> Conexión a MongoDB exitosa: mi_empresa
-- INICIO DE PRUEBAS CRUD --
Documento insertado, ID: 6989d1d4313f686255b6409b
Buscando empleado 'Carlos Perez':
Datos: {"_id": {"$oid": "6989d1d4313f686255b6409b"}, "nombre": "Carlos Perez", "puesto": "Desarrollador Junior", "salario": 25000, "activo": true}
Actualizando salario y puesto...
Dato actualizado: {"_id": {"$oid": "6989d1d4313f686255b6409b"}, "nombre": "Carlos Perez", "puesto": "Desarrollador Mid", "salario": 30000, "activo": true}
Eliminando empleado...
Empleados restantes con ese nombre: 0
>> Conexión cerrada.]
```

4. Conclusiones

La integración de Java con MongoDB mediante el driver oficial es un proceso directo y eficiente.

1. **Flexibilidad:** El uso de la clase `Document` permite manejar datos sin necesidad de definir esquemas rígidos (como tablas SQL) antes de programar.
2. **Entorno Dockerizado:** El uso de Docker facilita enormemente la preparación del entorno, eliminando problemas de configuración específicos del sistema operativo.
3. **Manejo de Recursos:** Es vital gestionar correctamente el cierre de conexiones (`connection.close()`) para evitar fugas de memoria en aplicaciones de larga duración.

5. Bibliografía

MongoDB Official Documentation:

<https://www.mongodb.com/docs/drivers/java/sync/current/>

Maven Repository:

<https://mvnrepository.com/artifact/org.mongodb/mongodb-driver-sync>